HTML

```
<!--apProject.html-->
<!-- Font taken from google fonts, icons just base UTF-8 text or google icons-->
<!-- Notification Bar js file separate site-wide notification system.-->
<!DOCTYPE html>
<html lang="en">

  <head>
    <meta charset="UTF-8">
    <meta name="robots" content="noindex, nofollow">
    <meta name="viewport"
      content="width=device-width, initial-scale=1.0, maximum-scale=1.0,
user-scalable=no, viewport-fit=cover">
    <title>AP Project Tam High 2026-Chase and Benjamin</title>
    <!--link to the CSS files-->
    <link rel="stylesheet" href="/projects/ApProject/apProject.css">
    <link rel="stylesheet" href="/notification-bars.css">
    <!--Use shared site manifest so all app metadata is centralized in one
file.-->
    <link rel="manifest" href="/manifest.json">
    <link rel="icon" href="/icons/icon-192x192.png">
    <!-- Font Import -->
    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
    <link
href="https://fonts.googleapis.com/css2?family=Cherry+Bomb+One&display=swap"
rel="stylesheet">
  </head>

  <body class="cherry-bomb-one-regular">
    <canvas id="gameArea">Game Area</canvas>
    <canvas id="webGlGameArea">WebGL Game Area</canvas>
    <h1 id="gameTitle" class="gameTitles">AP Project</h1>
    <button id="playButton" class="play-overlay" onclick="startGame()">▶</button>
    <button id="pauseButton" class="pause-button hidden" onclick="togglePause()">|
|</button>
    <button id="backButton" class="back-button" onclick="goBack()">Back</button>
    <button id="fullscreenButton" class="fs-button"
onclick="toggleFullScreen()">⛶</button>
    <button id="settingsButton" class="settings-button"
onclick="toggleSettings()">⚙️</button>
    <div id="settingsMenu" class="hidden settings-menu">
      <button id="closeSettingsMenu" class="close-menu"
onclick="toggleSettings()">×</button>
      <h2>Settings</h2>
```

```
      <!-- Performance Settings -->
      <div class="settings-section">
        <h3>Performance</h3>

        <label for="framerate" class="settings-label">Framerate:</label>
        <select id="framerate" class="settings-select">
          <option value="30">30 FPS</option>
          <option value="60" selected>60 FPS</option>
          <option value="120">120 FPS</option>
          <option value="144">144 FPS</option>
        </select>

        <label for="renderMode" class="settings-label">Render Mode:</label>
        <select id="renderMode" class="settings-select">
          <option value="cpu" selected>CPU</option>
          <option value="gpu">GPU</option>
          <option value="3D">GPU 3D</option>
        </select>
      </div>

      <!-- Graphics Settings -->
      <div class="settings-section">
        <h3>Graphics</h3>

        <label for="blockImages" class="settings-label">Block Textures:</label>
        <select id="blockImages" class="settings-select">
          <option value="on" selected>On</option>
          <option value="off">Off</option>
        </select>
        <p>Textures by <a href="https://piiixl.itch.io/textures" target="_blank"
rel="noopener noreferrer">piiixl</a>
        </p>
      </div>

      <!-- Audio Settings -->
      <div class="settings-section">
        <h3>Audio</h3>

        <label for="audioVolume" class="settings-label">Volume:</label>
        <div class="settings-slider-container">
          <input id="audioVolume" type="range" min="0" max="100" value="70"
class="settings-slider">
          <span id="volumeValue" class="settings-value">70%</span>
        </div>
```

```
        <label for="audioOutput" class="settings-label">Output:</label>
        <select id="audioOutput" class="settings-select">
          <option value="speakers" selected>Speakers</option>
          <option value="headphones">Headphones</option>
          <option value="mute">Mute</option>
        </select>
      </div>

      <!-- Control Settings -->
      <div class="settings-section">
        <h3>Controls</h3>

        <label for="jump" class="settings-label">Jump Keybind:</label>
        <button id="jump" class="keybind"></button>
        <label for="forward" class="settings-label">Forward Keybind:</label>
        <button id="forward" class="keybind"></button>
        <label for="backward" class="settings-label">Backward Keybind:</label>
        <button id="backward" class="keybind"></button>
        <label for="left" class="settings-label">Left Keybind:</label>
        <button id="left" class="keybind"></button>
        <label for="right" class="settings-label">Right Keybind:</label>
        <button id="right" class="keybind"></button>
        <label for="interact" class="settings-label">Interact Keybind:</label>
        <button id="interact" class="keybind"></button>
        <label for="previousLevel" class="settings-label">Previous Level
Keybind:</label>
        <button id="previousLevel" class="keybind"></button>
        <label for="nextLevel" class="settings-label">Next Level Keybind:</label>
        <button id="nextLevel" class="keybind"></button>
        <label for="previousLayer" class="settings-label">Previous Layer
Keybind:</label>
        <button id="previousLayer" class="keybind"></button>
        <label for="nextLayer" class="settings-label">Next Layer Keybind:</label>
        <button id="nextLayer" class="keybind"></button>
        <label for="respawn" class="settings-label">Respawn Keybind:</label>
        <button id="respawn" class="keybind"></button>

      </div>
    </div>
    <button id="levelEditorButton" class="level-editor-button"
onclick="toggleLevelEditor()">Level Editor</button>
    <div id="levelEditorMenu" class="hidden level-editor-menu">
      <button id="closeLevelEditorMenu" class="editor-button close-menu"
onclick="toggleLevelEditor()">×</button>
```

```
      <h2>Level Editor</h2>
      <button id="newLevelButton" class="editor-button"
onclick="createNewLevel()">New Level</button>
      <button id="loadLevelButton" class="editor-button"
onclick="loadLevel()">Load Level</button>
      <button id="exportLevelButton" class="editor-button"
onclick="exportLevel()">Export Level</button>
      <button id="importLevelButton" class="editor-button"
onclick="importLevel()">Import Level</button>
    </div>
    <div id="editLevelMenu" class="hidden edit-level-menu">
      <button class="close-menu" onclick="closeEditLevelMenu()">×</button>
      <h2>New Level</h2>
      <label for="newAreaButton">New Area:
        <button id="newAreaButton" class="editor-button"
onclick="newArea()">+</button>
      </label>
      <div id="areasContainer"></div>
      <div id="finalizeButtons">
        <button id="clearButton" class="editor-button"
onclick="clearEditLevelMenu()">Clear</button>
        <button id="submitButton" class="editor-button"
onclick="submitLevel()">Submit</button>
      </div>
    </div>
    <!-- Scripts (WOWZA)-->
    <script
src="https://cdnjs.cloudflare.com/ajax/libs/three.js/r128/three.min.js"></script>
    <script
src="https://cdn.jsdelivr.net/npm/three@0.128.0/examples/js/loaders/GLTFLoader.js"
></script>
    <script src="apProjectSettings.js"></script>
    <script src="apProjectLevelEditor.js"></script>
    <script src="apProjectLevelData.js"></script>
    <script src="apProjectLevelObjects.js"></script>
    <script src="apProjectLevelBuild.js"></script>
    <script src="apProjectThree.js"></script>
    <script src="apProject2dWebGl.js"></script>
    <script src="apProject.js"></script>
  </body>

</html>
```

CSS

```css
/* apProject.css */
html,
body {
	margin: 0;
	padding: 0;
	width: 100%;
	height: 100%;
	overflow: hidden;
	position: fixed;
	/* Prevents scrolling */
	touch-action: none;
	display: flex;
	background-color: 101 101 101;
}

/* Font Import */
.cherry-bomb-one-regular {
	font-family: "Cherry Bomb One", system-ui;
	font-display: swap;
	font-weight: 400;
	font-style: normal;
}

/* Overlay Buttons */
.play-overlay,
.fs-button,
.settings-button,
.pause-button,
.back-button,
.level-editor-button {
	position: absolute;
	cursor: pointer;
	border: none;
	border-radius: 8px;
	background: rgba(255, 189, 52, 0.85);
	z-index: 10;
	color: #333;
	transition: background 0.2s ease-in-out, transform 0.2s ease-in-out;
	font-weight: bold;
	box-shadow: 0 4px 6px rgba(149, 57, 0, 0.2);
}

.fs-button:hover,
.settings-button:hover,
.back-button:hover,
```

```css
.level-editor-button:hover {
	background: rgba(255, 189, 52, 1);
	transform: scale(1.05);
	box-shadow: 0 6px 12px rgba(149, 57, 0, 0.3);
}

.play-overlay {
	top: 50%;
	left: 50%;
	padding: 20px 30px;
	font-size: 2rem;
	transform: translate(-50%, -50%);
}

.play-overlay:hover {
	background: rgba(255, 189, 52, 1);
	transform: scale(1.05) translate(-50%, -50%);
	box-shadow: 0 6px 12px rgba(149, 57, 0, 0.3);
}

.pause-button {
	top: 15px;
	left: 50%;
	width: 48px;
	height: 48px;
	display: flex;
	justify-content: center;
	align-items: center;
	font-size: 1.5rem;
	transform: translateX(-50%);
}

.pause-button:hover {
	background: rgba(255, 189, 52, 1);
	transform: scale(1.05) translateX(-50%);
	box-shadow: 0 6px 12px rgba(149, 57, 0, 0.3);
}

.back-button {
	top: 15px;
	left: 15px;
	padding: 8px 16px;
	font-size: 1rem;
	display: flex;
	justify-content: center;
```

```
        align-items: center;
        gap: 6px;
}

.fs-button {
        bottom: 15px;
        right: 15px;
        width: 48px;
        height: 48px;
        display: flex;
        justify-content: center;
        align-items: center;
        font-size: 1.5rem;
}

.level-editor-button {
        bottom: 15px;
        left: 15px;
        padding: 8px 16px;
        font-size: 1rem;
        display: flex;
        justify-content: center;
        align-items: center;
        gap: 6px;
}

.settings-button {
        top: 15px;
        right: 15px;
        width: 48px;
        height: 48px;
        display: flex;
        justify-content: center;
        align-items: center;
        font-size: 1.5rem;
}

.settings-menu,
.level-editor-menu {
        position: fixed;
        background: #fffaf0;
        border: 3px dashed #ffe77b;
        padding: 1.5rem;
        box-shadow: 0 8px 24px rgba(149, 57, 0, 0.25);
        z-index: 11;
```

```
        max-width: 380px;
        max-height: 80vh;
        overflow-y: auto;
        bottom: 50%;
        right: 50%;
        transform: translate(50%, 50%);
        text-align: left;
        display: flex;
        flex-direction: column;
        border-radius: 8px;
        color: #333;
}

.settings-menu h2,
.level-editor-menu h2,
.edit-level-menu h2 {
        margin-top: 0;
        margin-bottom: 1rem;
        font-size: 1.5rem;
        color: var(--h2-light-mode-text-color);
        text-align: center;
        text-shadow: 1px 1px 2px rgba(149, 57, 0, 0.2);
}

.settings-section {
        margin-bottom: 1.5rem;
        padding-bottom: 1.5rem;
        border-bottom: 2px dashed #ffe77b;
}

.settings-section:last-child {
        border-bottom: none;
        margin-bottom: 0;
        padding-bottom: 0;
}

.settings-section h3,
.edit-level-menu h3 {
        margin-top: 0;
        margin-bottom: 1rem;
        font-size: 1.1rem;
        color: #333;
        text-transform: uppercase;
        letter-spacing: 1px;
        text-shadow: none;
```

```
}

.settings-label {
	display: block;
	margin-top: 0.75rem;
	margin-bottom: 0.5rem;
	font-weight: 600;
	color: #333;
	font-size: 0.95rem;
}

.settings-label::first-letter {
	text-transform: uppercase;
}

.settings-select {
	width: 100%;
	padding: 0.5rem 0.75rem;
	margin-bottom: 0.75rem;
	border: 2px solid #ffe77b;
	border-radius: 6px;
	background: white;
	color: #333;
	font-size: 0.95rem;
	cursor: pointer;
	transition: border-color 0.2s ease, background 0.2s ease;
	box-sizing: border-box;
}

.settings-select:hover {
	border-color: #ffb003;
	background: #fffef5;
}

.settings-select:focus {
	outline: none;
	border-color: #ff9800;
	box-shadow: 0 0 0 3px rgba(255, 152, 0, 0.1);
}

.settings-slider-container {
	display: flex;
	align-items: center;
	gap: 0.75rem;
	margin-bottom: 0.75rem;
```

```
}

.settings-slider {
	flex: 1;
	height: 8px;
	border-radius: 4px;
	background: linear-gradient(to right, #ff9800, #ffb74d);
	border: 1px solid #ffe77b;
	outline: none;
	-webkit-appearance: none;
	appearance: none;
	cursor: pointer;
}

.settings-value {
	min-width: 45px;
	text-align: right;
	font-weight: 600;
	color: #ff9800;
	font-size: 0.9rem;
}

.keybind,
.level-editor-menu button {
	width: 100%;
	padding: 0.6rem;
	margin-bottom: 0.75rem;
	border: 2px solid #ff9800;
	border-radius: 6px;
	background: white;
	color: #ff9800;
	font-weight: 600;
	font-size: 0.95rem;
	cursor: pointer;
	transition: all 0.2s ease;
	box-sizing: border-box;
	text-transform: uppercase;
	font: optional;
}

.keybind:hover,
.level-editor-menu button:hover,
.edit-level-menu button:hover {
	background: #fff8e7;
	border-color: #ffb74d;
```

```
        box-shadow: 0 2px 8px rgba(255, 152, 0, 0.15);
}

.keybind:active,
.level-editor-menu button:active,
.edit-level-menu button:active {
        background: #ffb74d;
        color: white;
        box-shadow: 0 2px 8px rgba(255, 152, 0, 0.3);
}

.hidden {
        display: none !important;
}

.gameTitles {
        position: absolute;
        top: 20px;
        left: 50%;
        transform: translateX(-50%);
        font-size: 2.5rem;
        color: #e9eeee;
        text-shadow: 2px 2px 4px rgba(0, 0, 0, 0.7);
}

.edit-level-menu {
        position: fixed;
        background: #fffaf0;
        border: 3px dashed #ffe77b;
        padding: 1.5rem;
        box-shadow: 0 8px 24px rgba(149, 57, 0, 0.25);
        z-index: 11;
        max-height: 80vh;
        overflow-y: auto;
        bottom: 50%;
        right: 50%;
        transform: translate(50%, 50%);
        text-align: left;
        display: flex;
        flex-direction: column;
        border-radius: 8px;
        color: #333;
}

.edit-level-menu button {
```

```
        width: fit-content;
        height: fit-content;
        padding: 0.15rem;
        margin-bottom: 0.75rem;
        border: 2px solid #ff9800;
        border-radius: 16px;
        background: white;
        color: #ff9800;
        font-weight: 600;
        font-size: 0.8rem;
        cursor: pointer;
        transition: all 0.2s ease;
        box-sizing: border-box;
        text-transform: uppercase;
        font: optional;
}

.close-menu {
        position: fixed;
        top: 5px;
        right: 5px;
        width: 28px !important;
        height: 28px !important;
        padding: 0;
        font-size: 0.9rem;
        display: flex;
        justify-content: center;
        align-items: center;
        border-radius: 50% !important;
}

.area {
        padding: 0.5rem;
        margin-bottom: 0.5rem;
        border: 4px dashed #ffe77b;
        border-radius: 8px;
        box-shadow: 0 8px 24px rgba(149, 57, 0, 0.25);
}

.area h3 {
        margin: 0;
}

.layer {
        padding: 0.5rem;
```

```
      margin-bottom: 0.5rem;
      border: 3px dashed #ffe77b;
      border-radius: 8px;
      box-shadow: 0 6px 18px rgba(149, 57, 0, 0.15);
}

.layer h4 {
      margin: 0;
}

.row {
      padding: 0.5rem;
      margin-bottom: 0.5rem;
      border: 2px dashed #ffe77b;
      border-radius: 8px;
      box-shadow: 0 4px 12px rgba(149, 57, 0, 0.1);
}

.row h5 {
      margin: 0;
      font-size: 0.9rem;
}

.tile {
      padding: 0.5rem;
      margin-bottom: 0.5rem;
      border: 1px dashed #ffe77b;
      border-radius: 8px;
      box-shadow: 0 2px 8px rgba(149, 57, 0, 0.05);
      display: flex;
      flex-direction: column;
      gap: 0.25rem;
}

.tile h6 {
      margin: 0;
      font-size: 0.85rem;
      color: #333;
}

.tileInput {
      width: auto;
      padding: 0.25rem;
      border: 1px solid #ffe77b;
      border-radius: 4px;
```

```
        background: white;
        color: #333;
        font-size: 0.85rem;
        transition: border-color 0.2s ease, background 0.2s ease;
        box-sizing: border-box;
}

#finalizeButtons {
        display: flex;
        justify-content: space-between;
        gap: 1rem;
        font-size: 0.5rem;
}

#clearButton {
        padding: 0.5rem 1rem;
        background: #ffb74d;
        color: white;
        border-color: #ff9800;
        margin: 0;
}

#submitButton {
        padding: 0.5rem 1rem;
        margin: 0;
}

#areasContainer {
        margin-bottom: 1.5rem;
}
```

JavaScript

```
//apProject.js
const canvas = document.getElementById("gameArea");
const ctx = canvas.getContext("2d");

const webGlCanvas = document.getElementById("webGlGameArea");
const gl = webGlCanvas.getContext("webgl2", { alpha: false}, {premultipliedAlpha:
false}, {antialias: false});

const webGl3dCanvas = document.getElementById("webGl3DCanvas");

function canvasDimensions(canvas) {
```

```
  canvas.style.width = "100dvw";
  canvas.style.height = "100dvh";

  // display:none canvases report 0 client size; keep a valid render size on
resize.
  const fallbackWidth = Math.floor(
    window.visualViewport?.width || window.innerWidth || 1,
  );
  const fallbackHeight = Math.floor(
    window.visualViewport?.height || window.innerHeight || 1,
  );
  const nextWidth = Math.max(1, canvas.clientWidth || fallbackWidth);
  const nextHeight = Math.max(1, canvas.clientHeight || fallbackHeight);

  if (canvas.width !== nextWidth) {
    canvas.width = nextWidth;
  }
  if (canvas.height !== nextHeight) {
    canvas.height = nextHeight;
  }
}

function resizeGameCanvases() {
  canvasDimensions(canvas);
  canvasDimensions(webGlCanvas);
  canvasDimensions(webGl3dCanvas);

  if (gl) {
    gl.viewport(0, 0, webGlCanvas.width, webGlCanvas.height);
  }
}

resizeGameCanvases();

if (window.visualViewport) {
  window.visualViewport.addEventListener("resize", resizeGameCanvases);
}
window.addEventListener("resize", resizeGameCanvases);
window.addEventListener("orientationchange", resizeGameCanvases);

function toggleFullScreen() {
  if (!document.fullscreenElement) {
    document.documentElement.requestFullscreen();
  } else {
    if (document.exitFullscreen) {
```

```
      document.exitFullscreen();
    }
  }
}

function toggleSettings() {
  const settingsMenu = document.getElementById("settingsMenu");
  if (settingsMenu.classList.contains("hidden")) {
    settingsMenu.classList.remove("hidden");
  } else {
    settingsMenu.classList.add("hidden");
  }
}

function toggleLevelEditor() {
  const levelEditorMenu = document.getElementById("levelEditorMenu");
  if (levelEditorMenu.classList.contains("hidden")) {
    levelEditorMenu.classList.remove("hidden");
  } else {
    levelEditorMenu.classList.add("hidden");
  }
}

// Game state
let gamePaused = false;
let gameRunning = false;

function togglePause() {
  gamePaused = !gamePaused;
  const pauseButton = document.getElementById("pauseButton");
  if (gamePaused) {
    pauseButton.textContent = "▶";
    pauseButton.title = "Resume Game";
  } else {
    pauseButton.textContent = "| |";
    pauseButton.title = "Pause Game";
  }
}

function goBack() {
  // Hide the game UI
  const gameTitle = document.getElementById("gameTitle");
  const playButton = document.getElementById("playButton");
  const pauseButton = document.getElementById("pauseButton");
```

```
  if (gameTitle) gameTitle.classList.remove("hidden");
  if (playButton) playButton.classList.remove("hidden");
  if (pauseButton) pauseButton.classList.add("hidden");

  // Reset game state
  gamePaused = false;
  gameRunning = false;

  // Go back to previous page
  window.history.back();
}

function backgroundColor(color) {
  ctx.fillStyle = color;
  ctx.fillRect(0, 0, canvas.width, canvas.height);
}

backgroundColor("dimgray");
webGlBackgroundColor(105, 105, 105, 1);
sceneBackgroundColor(105, 105, 105);

// Code found online on detecting if a character is a letter
function isLetter(char) {
  return /^[a-z]$/i.test(char);
}

const controller = {
  jump: { pressed: false, key: [" "] },
  forward: { pressed: false, key: ["W", "w"] },
  backward: { pressed: false, key: ["S", "s"] },
  left: { pressed: false, key: ["A", "a"] },
  right: { pressed: false, key: ["D", "d"] },
  interact: { pressed: false, key: ["E", "e"] },
  previousLevel: { pressed: false, key: ["ArrowLeft"] },
  nextLevel: { pressed: false, key: ["ArrowRight"] },
  previousLayer: { pressed: false, key: ["ArrowUp"] },
  nextLayer: { pressed: false, key: ["ArrowDown"] },
  respawn: { pressed: false, key: ["K", "k"] },
};
initializeSettings();

function updateKeybind(keybind, newKey) {
  for (const controllerKey in controller) {
    if (
      controller[controllerKey].key.includes(newKey) &&
```

```
      controllerKey !== keybind
    ) {
      // Clear the conflicting keybind
      controller[controllerKey].key = [];
      const button = document.getElementById(controllerKey);
      button.textContent = "";
    }
  }
  if (isLetter(newKey)) {
    controller[keybind].key = [newKey.toUpperCase(), newKey.toLowerCase()];
  } else {
    controller[keybind].key = [newKey];
  }
}

const player = new Player(45, "images/Mario.png");

let currentLevel = 0;
let currentArea = 0;
let levelSwitchCooldown = false;
let layerSwitchCooldown = false;
let longestHorizontalLength;
let longestVerticalLength;
let activeLevelData = [];

function removeAllObjects() {
  for (let i = activeLevelData[currentArea].length - 1; i >= 0; i--) {
    const layer = activeLevelData[currentArea][i];
    for (let j = layer.length - 1; j >= 0; j--) {
      const row = layer[j];
      for (let k = row.length - 1; k >= 0; k--) {
        const obj = row[k];
        obj.remove();
      }
    }
  }
}

function updateLevelData() {
  currentArea = 0;
  activeLevelData = [];
  for (let i = 0; i < levels[currentLevel].length; i++) {
    activeLevelData.push([]);
    for (let j = 0; j < levels[currentLevel][i].length; j++) {
      activeLevelData[i].push(
```

```
        convertToObjects(levels[currentLevel][i][j], j, i, player),
      );
    }
  }
  longestHorizontalLength = Math.max.apply(
    null,
    activeLevelData[currentArea].map((layer) =>
      Math.max.apply(
        null,
        layer.map((row) => row.length),
      ),
    ),
  );
  longestVerticalLength = Math.max.apply(
    null,
    activeLevelData[currentArea].map((layer) => layer.length),
  );
}

let times = [];

function gameLoop() {
  // FPS calculation
  const currentTime = performance.now();

  while (times.length > 0 && times[0] <= currentTime - 1000) {
    times.shift();
  }
  times.push(currentTime);
  fps = times.length;


  if (!webGl && !webGl3d) {
    backgroundColor("lightblue");
  } else if (webGl) {
    if (!shadersInitialized) {
      initializeWebGL();
      shadersInitialized = true;
    }
    requestAnimationFrame(webGLRender2D);
  } else if (webGl3d) {
    renderScene();
  }
  for (let i = 0; i < activeLevelData[currentArea].length; i++) {
    buildLevel(activeLevelData[currentArea][i]);
```

```
    if (player.layer == i) {
      player.drawPlayer();
    }
  }

  // Skip game logic if paused, but still update the display
  if (gamePaused) {
    requestAnimationFrame(gameLoop);
    return;
  } else {
    ctx.globalAlpha = 1;
    player.move(controller);
    player.grounded = false;
    player.currentGroundBlock = null;
    player.insideBlock = null;
    let areaChanged = false;
    try {
      const layerData = activeLevelData[currentArea][player.layer];
      for (let i = 0; i < layerData.length; i++) {
        for (let j = 0; j < layerData[i].length; j++) {
          const levelObject = layerData[i][j];
          if (!levelObject) {
            continue;
          }

          if (levelObject.isBlock) {
            const block = levelObject;
            if (block.solid) {
              if (
                Math.abs(player.x - block.x + player.globalOffsetX) < 100 &&
                Math.abs(player.y - block.y + player.globalOffsetY) < 100
              ) {
                const wasInsideThisBlock = player.isOverlappingObject(block,
true);
                const insideThisBlock = player.insideBlockDetection(block);
                const shouldIgnoreCollision = wasInsideThisBlock &&
insideThisBlock;

                if (!shouldIgnoreCollision) {
                  player.resolveSolidCollision(block);
                }
              }
            } else if (player.insideBlockDetection(block)) {
              if (controller.interact.pressed) {
                if (
```

```
                block.type === "areaDoor" ||
                block.type === "areaDoorBottom" ||
                block.type === "areaDoorTop"
              ) {
                removeAllObjects();
                currentArea = Number(block.doorArea);
                player.spawn();
                areaChanged = true;
                break;
              }
            }
            if (block.type === "spike") {
              player.die();
              break;
            }
          }
        } else if (levelObject.isEnemy) {
          const enemy = levelObject;
          enemy.update();
          if (player.isOverlappingObject(enemy, true)) {
            player.die();
            break;
          }
        }

        if (areaChanged) {
          break;
        }
      }
    }
  } catch (error) {
    console.warn("[APProject] Collision iteration failed", {
      currentArea,
      playerLayer: player.layer,
      playerX: player.x,
      playerY: player.y,
      offsetX: player.globalOffsetX,
      offsetY: player.globalOffsetY,
      error,
    });
  }
}

player.gameBoundaryDetection();
if (controller.previousLevel.pressed) {
```

```
  if (!levelSwitchCooldown) {
    if (currentLevel > 0) {
      currentLevel--;
      levelSwitchCooldown = true;
      setTimeout(() => {
        levelSwitchCooldown = false;
      }, 250);
      removeAllObjects();
      updateLevelData();
      player.spawn();
    }
  }
}
if (controller.nextLevel.pressed) {
  if (!levelSwitchCooldown) {
    if (currentLevel < levels.length - 1) {
      currentLevel++;
      levelSwitchCooldown = true;
      setTimeout(() => {
        levelSwitchCooldown = false;
      }, 250);
      removeAllObjects();
      updateLevelData();
      player.spawn();
    }
  }
}
if (controller.nextLayer.pressed) {
  if (!layerSwitchCooldown) {
    layerSwitchCooldown = true;
    setTimeout(() => {
      layerSwitchCooldown = false;
    }, 250);
    if (
      player.layer <
      activeLevelData[currentArea][player.layer].length - 1
    ) {
      player.layer++;
    }
    if (player.layer >= activeLevelData[currentArea].length) {
      player.layer = activeLevelData[currentArea].length - 1;
    }
  }
}
if (controller.previousLayer?.pressed) {
```

```
    if (!layerSwitchCooldown) {
      layerSwitchCooldown = true;
      setTimeout(() => {
        layerSwitchCooldown = false;
      }, 250);
      if (player.layer > 0) {
        player.layer--;
      }
      if (player.layer <= 0) {
        player.layer = 0;
      }
    }
  }

  if (controller.respawn.pressed) {
    player.die();
  }

  requestAnimationFrame(gameLoop);
}

function startGame() {
  const gameTitle = document.getElementById("gameTitle");
  gameTitle.classList.add("hidden");
  const playButton = document.getElementById("playButton");
  playButton.classList.add("hidden");
  const pauseButton = document.getElementById("pauseButton");
  pauseButton.classList.remove("hidden");

  gameRunning = true;
  gamePaused = false;
  pauseButton.textContent = "| |";
  pauseButton.title = "Pause Game";

  if (webGl) {
    initializeWebGL();
    canvas.style.display = "none";
    webGlCanvas.style.display = "block";
    webGl3dCanvas.style.display = "none";
  } else if (webGl3d) {
    canvas.style.display = "none";
    webGlCanvas.style.display = "none";
    webGl3dCanvas.style.display = "block";
  }
```

```
  updateLevelData();
  player.spawn();
  gameLoop();
}

window.document.addEventListener("keydown", function (e) {
  for (const controllerKey in controller) {
    if (controller[controllerKey].key.includes(e.key)) {
      controller[controllerKey].pressed = true;
    }
  }
});

window.document.addEventListener("keyup", function (e) {
  for (const controllerKey in controller) {
    if (controller[controllerKey].key.includes(e.key)) {
      controller[controllerKey].pressed = false;
    }
  }
});
```

JavaScript

```
//apProjectLevelObjects.js
function getBlockTexturePath(index) {
  return `images/Block-assets/texture_16px ${index}.png`;
}

class Player {
  constructor(size, imageSrc) {
    this.x = 0;
    this.y = 0;
    this.z = 0;
    this.baseSize = size;
    this.size = size;
    this.image = new Image();
    this.image.src = imageSrc;

    this.r = 255;
    this.g = 0;
    this.b = 255;

    this.width = null;
```

```
  this.spawnValues = [];

  this.velX = 0;
  this.velY = 0;
  this.velZ = 0;
  this.prevY = 0;
  this.prevX = 0;
  this.speed = 0.3;
  this.gravity = 0.5;
  this.jumpStrength = -11;
  this.jumps = 0;
  this.maxAirJumps = 1;
  this.grounded = false;
  this.maxFallSpeed = 15;
  this.maxVelX = 5;
  this.maxVelY = 15;
  this.currentGroundBlock = null;
  this.insideBlock = null;

  this.globalOffsetX = 0;
  this.globalOffsetY = 0;
  this.prevGlobalOffsetX = 0;
  this.prevGlobalOffsetY = 0;

  this.direction = "right";

  this.layer = 0;

  this.dead = false;
  this.canMove = true;
  this.deathRespawnTimeout = null;

  this.alpha = 1;

  this.sizeChangeCooldown = false;
  this.state = 0;
}

spawn() {
  if (this.deathRespawnTimeout) {
    clearTimeout(this.deathRespawnTimeout);
    this.deathRespawnTimeout = null;
  }

  this.dead = false;
```

```
    this.canMove = true;
    this.alpha = 1;

    if (webGl3d) {
      ambientLight.color.set(0x404040);
    }

    this.x = this.spawnValues[currentArea][0];
    this.y = this.spawnValues[currentArea][1];
    if (this.state === 1) {
      this.y -= this.baseSize * 1.5;
    } else if (this.state === -1) {
      this.y += this.baseSize * 0.5;
    }
    this.layer = this.spawnValues[currentArea][2];
    this.z = this.layer * 50;
    this.globalOffsetX = this.spawnValues[currentArea][3];
    this.globalOffsetY = this.spawnValues[currentArea][4];

    this.velX = 0;
    this.velY = 0;
    this.velZ = 0;
    this.prevY = this.y;
    this.prevX = this.x;
  }

  die() {
    if (this.dead) {
      return;
    }

    this.dead = true;
    this.canMove = false;
    this.velX = 0;
    this.velY = 0;
    this.velZ = 0;

    // Keep respawn independent from rendering so empty/unrendered layers still
recover.
    this.deathRespawnTimeout = setTimeout(() => {
      if (this.dead) {
        this.spawn();
      }
    }, 1800);
  }
```

```
drawPlayer() {
  this.width = Math.floor(this.size * (this.image.width / this.image.height));
  if (this.dead) {
    this.alpha -= 0.01;
    if (webGl3d) {
      ambientLight.color.set(`rgb(${Math.floor(105 * this.alpha)}, 0, 0)`);
      sceneBackgroundColor(Math.floor(105 * this.alpha), 0, 0);
    }
    if (this.alpha <= 0) {
      this.spawn();
    }
  }
  if (this.image.complete && this.image.width > 0 && !webGl) {
    ctx.globalAlpha = this.alpha;
    if (this.direction === "right") {
      ctx.drawImage(
        this.image,
        Math.round(this.x),
        Math.round(this.y),
        this.width,
        this.size,
      );
    } else {
      ctx.save();
      ctx.scale(-1, 1);
      ctx.drawImage(
        this.image,
        Math.round(-this.x - this.width),
        Math.round(this.y),
        this.width,
        this.size,
      );
      ctx.restore();
    }
    ctx.globalAlpha = 1;
  } else if (webGl && this.direction === "right") {
    renderImage(
      this.image,
      Math.round(this.x),
      Math.round(this.y),
      this.width,
      this.size,
      this.alpha,
    );
```

```
  } else if (webGl && this.direction === "left") {
    renderImage(
      this.image,
      Math.round(this.x) + this.width,
      Math.round(this.y),
      this.width * -1,
      this.size,
      this.alpha,
    );
  }
}

move(controller) {
  if (!webGl3d) {
    this.z = this.layer * 50;
  }
  if (this.canMove) {
    if (!webGl3d) {
      if (controller.right.pressed) {
        this.direction = "right";
        if (this.velX < this.maxVelX) this.velX += this.speed;
      }
      if (controller.left.pressed) {
        this.direction = "left";
        if (this.velX > -this.maxVelX) this.velX -= this.speed;
      }
    }

    if (
      !webGl3d &&
      this.currentGroundBlock != null &&
      !controller.right.pressed &&
      !controller.left.pressed
    ) {
      this.velX *= this.currentGroundBlock.friction;
    }
    if (this.insideBlock !== null) {
      if (this.insideBlock.type === "cobweb") {
        this.velX *= this.insideBlock.friction;
        if (webGl3d) {
          this.velZ *= this.insideBlock.friction;
        }
      }
    }
```

```
if (controller.jump.pressed && this.grounded && !this.jumpCooldown) {
  this.velY = this.jumpStrength;
  this.grounded = false;
  setTimeout(() => {
    this.jumpCooldown = false;
  }, 200);
  this.jumpCooldown = true;
} else if (
  controller.jump.pressed &&
  this.jumps >= 1 &&
  !this.jumpCooldown
) {
  this.velY = this.jumpStrength * 0.75;
  this.jumps -= 1;
  setTimeout(() => {
    this.jumpCooldown = false;
  }, 200);
  this.jumpCooldown = true;
}

this.prevY = this.y;
this.prevX = this.x;

if (!this.grounded) {
  this.velY += this.gravity;
  if (this.insideBlock !== null) {
    this.velY *= this.insideBlock.verticalFriction;
  }
  if (this.velY > this.maxFallSpeed) {
    this.velY = this.maxFallSpeed;
  }
}

this.snapVelocityToZero("velX");
this.snapVelocityToZero("velY");

// Store previous offsets before scrolling
this.prevGlobalOffsetX = this.globalOffsetX;
this.prevGlobalOffsetY = this.globalOffsetY;

this.scrolling();

if (webGl3d) {
  let movementVector = new THREE.Vector3();
  if (controller.forward.pressed) {
```

```
    const forward = new THREE.Vector3();
    camera.getWorldDirection(forward);
    forward.y = 0;
    forward.normalize();
    movementVector.add(forward);
  }
  if (controller.backward.pressed) {
    const backward = new THREE.Vector3();
    camera.getWorldDirection(backward);
    backward.y = 0;
    backward.normalize();
    movementVector.sub(backward);
  }
  if (controller.left.pressed) {
    const left = new THREE.Vector3();
    camera.getWorldDirection(left);
    left.y = 0;
    left.cross(camera.up);
    left.normalize();
    movementVector.sub(left);
  }
  if (controller.right.pressed) {
    const right = new THREE.Vector3();
    camera.getWorldDirection(right);
    right.y = 0;
    right.cross(camera.up);
    right.normalize();
    movementVector.add(right);
  }
  movementVector.normalize();
  movementVector.multiplyScalar(this.speed);
  movementVector.x += this.velX;
  movementVector.z += this.velZ;
  movementVector.clampLength(0, this.maxVelX);
  this.velX = movementVector.x;
  this.velZ = movementVector.z;
  if (
    this.currentGroundBlock != null &&
    !controller.right.pressed &&
    !controller.left.pressed &&
    !controller.forward.pressed &&
    !controller.backward.pressed
  ) {
    this.velX *= this.currentGroundBlock.friction;
    this.velZ *= this.currentGroundBlock.friction;
```

```
      }
      this.snapVelocityToZero("velX");
      this.snapVelocityToZero("velZ");
      this.x += this.velX;
      this.z += this.velZ;
      this.layer = Math.round(this.z / 50);
    }
  }
}

snapVelocityToZero(axis, threshold = 0.1) {
  if (Math.abs(this[axis]) <= threshold) {
    this[axis] = 0;
  }
}

scrolling() {
  let longestHorizontalLayerLength = 0;
  let longestVerticalLayerLength = 0;
  try {
    for (let i = 0; i < activeLevelData[currentArea].length; i++) {
      if (
        activeLevelData[currentArea][i].length > longestHorizontalLayerLength
      ) {
        longestHorizontalLayerLength = activeLevelData[currentArea][i].length;
      }
      if (
        activeLevelData[currentArea][i].length > longestVerticalLayerLength
      ) {
        longestVerticalLayerLength = activeLevelData[currentArea][i].length;
      }
    }
  } catch (error) {
    console.warn("[APProject] Failed to read level bounds for scrolling", {
      currentArea,
      playerLayer: this.layer,
      error,
    });
    if (!webGl3d) this.x += this.velX;
    this.y += this.velY;
    return;
  }
  if (
    (this.x >= gameArea.width / 2 || this.globalOffsetX > 0) &&
    this.globalOffsetX < longestHorizontalLayerLength * 50 - gameArea.width &&
```

```
    longestHorizontalLayerLength * 50 > gameArea.width
  ) {
    this.globalOffsetX += this.velX;
    if (this.globalOffsetX <= 0) {
      this.globalOffsetX = 0;
      if (!webGl3d) this.x += this.velX;
    }
  } else {
    if (!webGl3d) this.x += this.velX;
  }
  if (
    this.globalOffsetX >=
      longestHorizontalLayerLength * 50 - gameArea.width &&
    longestHorizontalLayerLength * 50 > gameArea.width
  ) {
    if (this.x >= gameArea.width / 2) {
      this.globalOffsetX = longestHorizontalLayerLength * 50 - gameArea.width;
    } else if (this.x < gameArea.width / 2) {
      this.globalOffsetX += this.velX;
    }
  }

  if (
    (this.y >= gameArea.height / 2 || this.globalOffsetY > 0) &&
    this.globalOffsetY < longestVerticalLayerLength * 50 - gameArea.height &&
    longestVerticalLayerLength * 50 > gameArea.height
  ) {
    this.globalOffsetY += this.velY;
    if (this.globalOffsetY <= 0) {
      this.globalOffsetY = 0;
      this.y += this.velY;
    }
  } else {
    this.y += this.velY;
  }
  if (
    this.globalOffsetY >= longestVerticalLayerLength * 50 - gameArea.height &&
    longestVerticalLayerLength * 50 > gameArea.height
  ) {
    if (this.y >= gameArea.height / 2) {
      this.globalOffsetY = longestVerticalLayerLength * 50 - gameArea.height;
    } else if (this.y < gameArea.height / 2) {
      this.globalOffsetY += this.velY;
    }
  }
```

```
}

getWorldBounds(usePreviousFrame = false) {
  const offsetX = usePreviousFrame
    ? this.prevGlobalOffsetX
    : this.globalOffsetX;
  const offsetY = usePreviousFrame
    ? this.prevGlobalOffsetY
    : this.globalOffsetY;
  const playerX = usePreviousFrame ? this.prevX : this.x;
  const playerY = usePreviousFrame ? this.prevY : this.y;

  const left = playerX + offsetX;
  const top = playerY + offsetY;

  return {
    left,
    right: left + this.width,
    top,
    bottom: top + this.size,
  };
}

resolveSolidCollision(object) {
  if (!object || !object.solid) {
    return;
  }

  const previousBounds = this.getWorldBounds(true);
  let currentBounds = this.getWorldBounds(false);

  const blockLeft = object.x;
  const blockRight = object.x + object.width;
  const blockTop = object.y;
  const blockBottom = object.y + object.height;

  const overlapsY =
    currentBounds.bottom > blockTop && currentBounds.top < blockBottom;

  if (overlapsY) {
    const enteredLeftFace =
      previousBounds.right <= blockLeft &&
      currentBounds.right > blockLeft &&
      this.velX > 0;
    const enteredRightFace =
```

```
      previousBounds.left >= blockRight &&
      currentBounds.left < blockRight &&
      this.velX < 0;

    if (enteredLeftFace) {
      this.globalOffsetX = this.prevGlobalOffsetX;
      this.x = blockLeft - this.width - this.globalOffsetX;
      this.velX = 0;
    } else if (enteredRightFace) {
      this.globalOffsetX = this.prevGlobalOffsetX;
      this.x = blockRight - this.globalOffsetX;
      this.velX = 0;
    }
  }

  currentBounds = this.getWorldBounds(false);
  const overlapsX =
    currentBounds.right > blockLeft && currentBounds.left < blockRight;

  if (overlapsX) {
    const landedOnTop =
      previousBounds.bottom <= blockTop &&
      currentBounds.bottom >= blockTop &&
      this.velY >= 0;
    const hitUnderside =
      previousBounds.top >= blockBottom &&
      currentBounds.top <= blockBottom &&
      this.velY < 0;

    if (landedOnTop) {
      this.grounded = true;
      this.jumps = this.maxAirJumps;
      this.currentGroundBlock = object;
      if (
        this.currentGroundBlock.temporary &&
        this.currentGroundBlock.timeToDisappear === undefined
      ) {
        object.startTempDisappear();
      }
      this.velY = 0;
      this.y = object.y - this.size - this.globalOffsetY;
    } else if (hitUnderside) {
      this.velY = 0;
      this.y = blockBottom - this.globalOffsetY;
    }
```

```
  }
}

gameBoundaryDetection() {
  if (this.x <= 0 && !webGl3d) {
    this.x = 0;
    this.velX = 0;
  }
  if (this.x + this.width > gameArea.width && !webGl3d) {
    this.x = gameArea.width - this.width;
    this.velX = 0;
  }
  if (this.y <= 0 && !webGl3d) {
    this.y = 0;
    this.velY = 0;
  }
  if (this.y + this.size >= gameArea.height) {
    this.spawn();
  }
}

isOverlappingObject(object, usePreviousFrame = false) {
  const offsetX = usePreviousFrame
    ? this.prevGlobalOffsetX
    : this.globalOffsetX;
  const offsetY = usePreviousFrame
    ? this.prevGlobalOffsetY
    : this.globalOffsetY;
  const playerX = usePreviousFrame ? this.prevX : this.x;
  const playerY = usePreviousFrame ? this.prevY : this.y;

  const playerLeft = playerX + offsetX;
  const playerRight = playerLeft + this.width;
  const playerTop = playerY + offsetY;
  const playerBottom = playerTop + this.size;

  const objectLeft = object.x;
  const objectRight = object.x + object.width;
  const objectTop = object.y;
  const objectBottom = object.y + object.height;

  return (
    playerRight > objectLeft &&
    playerLeft < objectRight &&
    playerBottom > objectTop &&
```

```
      playerTop < objectBottom
    );
  }

  insideBlockDetection(block) {
    const isInside = this.isOverlappingObject(block);

    if (isInside) {
      this.insideBlock = block;
    }

    return isInside;
  }
}

let defaultFriction = 0.8;

const blockTypes = {
  basic: {
    colorR: 165,
    colorG: 42,
    colorB: 42,
    solid: true,
    temporary: false,
    friction: defaultFriction,
    verticalFriction: 1,
    textureIndex: 66,
    texture: "images/Block-assets/texture_16px 66.png",
  },
  temporary: {
    colorR: 128,
    colorG: 128,
    colorB: 128,
    solid: true,
    temporary: true,
    disappearTime: 30,
    reappearTime: 120,
    friction: defaultFriction,
    verticalFriction: 1,
    textureIndex: 113,
    texture: "images/Block-assets/texture_16px 113.png",
  },
  permanentTemporary: {
    colorR: 105,
    colorG: 105,
```

```
  colorB: 105,
  solid: true,
  temporary: true,
  disappearTime: 45,
  reappearTime: undefined,
  friction: defaultFriction,
  verticalFriction: 1,
  textureIndex: 144,
  texture: "images/Block-assets/texture_16px 144.png",
},
ice: {
  colorR: 0,
  colorG: 255,
  colorB: 255,
  solid: true,
  temporary: false,
  friction: 0.99,
  verticalFriction: 1,
  textureIndex: 201,
  texture: "images/Block-assets/texture_16px 201.png",
},
slow: {
  colorR: 0,
  colorG: 100,
  colorB: 0,
  solid: true,
  temporary: false,
  friction: 0.75,
  verticalFriction: 1,
  textureIndex: 206,
  texture: "images/Block-assets/texture_16px 206.png",
},
cobweb: {
  colorR: 255,
  colorG: 255,
  colorB: 255,
  solid: false,
  temporary: false,
  friction: 0.5,
  verticalFriction: 0.25,
  textureIndex: 339,
  texture: "images/Block-assets/texture_16px 339.png",
},
spike: {
  colorR: 0,
```

```
    colorG: 0,
    colorB: 0,
    solid: false,
    temporary: false,
    friction: 1,
    verticalFriction: 1,
    textureIndex: 487,
    texture: "images/Block-assets/texture_16px 487.png",
  },
  areaDoorBottom: {
    colorR: 255,
    colorG: 215,
    colorB: 0,
    solid: false,
    temporary: false,
    friction: 1,
    verticalFriction: 1,
    textureIndex: 590,
    texture: "images/Block-assets/texture_16px 591.png",
  },
  areaDoorTop: {
    colorR: 255,
    colorG: 215,
    colorB: 0,
    solid: false,
    temporary: false,
    friction: 1,
    verticalFriction: 1,
    textureIndex: 591,
    texture: "images/Block-assets/texture_16px 590.png",
  },
};

class Block {
  static texturePool = [];
  static texturesLoaded = false;

  static getTexture(index) {
    if (!Block.texturePool[index - 1]) {
      const image = new Image();
      try {
        image.src = getBlockTexturePath(index);
      } catch (error) {
        console.warn("[APProject] Failed to load block texture", {
          textureIndex: index,
```

```
          error,
        });
      }
      Block.texturePool[index - 1] = image;
    }

    return Block.texturePool[index - 1];
  }

  static initializeTextures() {
    if (Block.texturesLoaded) {
      return;
    }

    const textureIndexes = [
      ...new Set(
        Object.values(blockTypes)
          .map((blockType) => blockType.textureIndex)
          .filter((textureIndex) => Number.isInteger(textureIndex)),
      ),
    ];

    textureIndexes.forEach((index) => {
      Block.getTexture(index);
    });

    Block.texturesLoaded = true;
  }

  constructor(x, y, width, height, layer, type, doorArea) {
    Block.initializeTextures();

    this.x = x;
    this.y = y;
    this.width = width;
    this.height = height;
    this.layer = layer;
    this.type = type;
    this.colorR = blockTypes[type].colorR;
    this.colorG = blockTypes[type].colorG;
    this.colorB = blockTypes[type].colorB;
    this.alpha = 1;
    this.color = `rgba(${this.colorR}, ${this.colorG}, ${this.colorB},
${this.alpha})`;
    this.solid = blockTypes[type].solid;
```

```
    this.temporary = blockTypes[type].temporary;
    this.timeToDisappear;
    this.timeToReappear;
    this.friction = blockTypes[this.type].friction;
    this.verticalFriction = blockTypes[this.type].verticalFriction;
    this.textureIndex = blockTypes[this.type].textureIndex || 1;
    this.visible = true;
    this.doorArea = null;
    if (this.type === "areaDoorBottom" || this.type === "areaDoorTop") {
      this.doorArea = doorArea;
    }

    this.cube = null;

    this.isBlock = true;
    this.isEnemy = false;
    this.texture = Block.getTexture(this.textureIndex);

    this.setTexture = false;
  }
  colorReset() {
    this.color = `rgba(${this.colorR}, ${this.colorG}, ${this.colorB},
${this.alpha})`;
  }
  drawBlock() {
    if (!webGl3d) {
      this.alpha = Math.max(
        0.1,
        1 - Math.abs(player.layer - this.layer) * 0.85,
      );
    } else {
      this.alpha = 1;
    }
    const blockTypeData = blockTypes[this.type];

    if (this.temporary && this.timeToDisappear !== undefined) {
      this.timeToDisappear -= 1;
      this.alpha = this.timeToDisappear / blockTypeData.disappearTime;
      this.colorReset();
      if (this.timeToDisappear <= 0) {
        this.tempDisappear();
      }
    }
    if (this.temporary && this.timeToReappear !== undefined) {
      this.timeToReappear -= 1;
```

```
    this.colorReset();
    if (this.timeToReappear <= 0) {
      this.tempReappear();
    }
  }

  if (this.layer !== player.layer) {
    let color = Math.max(this.colorR, this.colorG, this.colorB);
    color = Math.floor(
      color * (1 - Math.abs(player.layer - this.layer) * 0.85),
    );
    this.color = `rgba(${color}, ${color}, ${color}, ${this.alpha})`;
  } else {
    this.colorReset();
  }

  if (this.alpha <= 0 || this.timeToReappear !== undefined) {
    this.visible = false;
  } else {
    this.visible = true;
  }

  if (
    !webGl3d &&
    (Math.round(this.x - player.globalOffsetX + this.width) < 0 ||
      Math.round(this.x - player.globalOffsetX) > gameArea.width ||
      Math.round(this.y - player.globalOffsetY + this.height) < 0 ||
      Math.round(this.y - player.globalOffsetY) > gameArea.height)
  ) {
    this.visible = false;
  }

  const shouldUseTextures =
    window.gameSettings && window.gameSettings.blockImages !== false;

  // Skip rendering if not visible or fully transparent
  if (this.visible && this.alpha > 0) {
    const renderX = Math.round(this.x - player.globalOffsetX);
    const renderY = Math.round(this.y - player.globalOffsetY);
    if (shouldUseTextures && this.texture && !webGl && !webGl3d) {
      const previousSmoothing = ctx.imageSmoothingEnabled;
      ctx.imageSmoothingEnabled = false;
      ctx.globalAlpha = this.alpha;
      ctx.drawImage(this.texture, renderX, renderY, this.width, this.height);
      ctx.globalAlpha = 1;
```

```
      ctx.imageSmoothingEnabled = previousSmoothing;
    } else if (!webGl && !webGl3d) {
      ctx.fillStyle = this.color;
      ctx.fillRect(renderX, renderY, this.width, this.height);
    } else if (webGl && shouldUseTextures && this.texture && !webGl3d) {
      renderImage(
        this.texture,
        renderX,
        renderY,
        this.width,
        this.height,
        this.alpha,
      );
    } else if (webGl && !webGl3d) {
      let index = Object.keys(blockTypes).indexOf(this.type);
      renderRect(
        renderX,
        renderY,
        this.width,
        this.height,
        index,
        this.alpha,
      );
    } else if (webGl3d) {
      if (!this.cube) {
        this.cube = createColoredCube(this.colorR, this.colorG, this.colorB);
      } else {
        this.cube.visible = true;
      }
      if (shouldUseTextures && this.texture && !this.setTexture) {
        setTexture(this.cube, this.texture);
        this.setTexture = true;
      } else if (!shouldUseTextures) {
        setColor(this.cube, this.colorR, this.colorG, this.colorB);
        this.setTexture = false;
      }
      this.cube.position.x = renderX;
      this.cube.position.y = renderY;
      this.cube.position.z = this.layer * 50;
      setOpacity(this.cube, this.alpha);
    }
  } else if (webGl3d && this.cube) {
    this.visible = true;
  }
```

```
    if (!webGl3d) {
      this.alpha = Math.max(
        0.1,
        1 - Math.abs(player.layer - this.layer) * 0.85,
      );
    } else {
      this.alpha = 1;
    }
  }
  startTempDisappear() {
    this.timeToDisappear = blockTypes[this.type].disappearTime;
  }
  tempDisappear() {
    this.solid = false;
    this.timeToDisappear = undefined;
    this.timeToReappear = blockTypes[this.type].reappearTime;
    this.visible = false;
  }
  tempReappear() {
    this.solid = true;
    this.alpha = 1;
    this.colorReset();
    this.timeToReappear = undefined;
    this.visible = true;
  }
  remove() {
    if (webGl3d && this.cube) {
      removeObject(this.cube);
      this.cube = null;
    }
  }
}

const enemyTypes = {
  0: {
    name: "Horizontal Patroller",
    width: 50,
    height: 50,
    speed: 1,
    movement: "horizontalPatrol",
    colorR: 255,
    colorG: 0,
    colorB: 0,
  },
  1: {
```

```
    name: "Vertical Patroller",
    width: 50,
    height: 50,
    speed: 1,
    movement: "verticalPatrol",
    colorR: 0,
    colorG: 0,
    colorB: 255,
  },
};

class Enemy {
  constructor(x, y, type, layer) {
    this.x = x;
    this.y = y;
    this.layer = layer;
    this.type = type;
    this.width = enemyTypes[type].width;
    this.height = enemyTypes[type].height;
    this.speed = enemyTypes[type].speed;
    this.direction = 1;
    this.movement = enemyTypes[type].movement;
    /*this.image = new Image();
    this.image.src = enemyTypes[type].imageSrc;*/

    this.colorR = enemyTypes[type].colorR;
    this.colorG = enemyTypes[type].colorG;
    this.colorB = enemyTypes[type].colorB;
    this.alpha = 1;
    this.color = `rgba(${this.colorR}, ${this.colorG}, ${this.colorB},
${this.alpha})`;

    this.isBlock = false;
    this.isEnemy = true;

    this.cube = null;
    this.model = "./3D-Models/Kieran.glb";
  }

  colorReset() {
    this.color = `rgba(${this.colorR}, ${this.colorG}, ${this.colorB},
${this.alpha})`;
  }

  drawEnemy() {
```

```
if (!webGl3d) {
  this.alpha = Math.max(
    0.1,
    1 - Math.abs(player.layer - this.layer) * 0.85,
  );
} else if (webGl3d) {
  this.alpha = 1;
}

if (this.layer !== player.layer) {
  let color = Math.max(this.colorR, this.colorG, this.colorB);
  color = Math.floor(
    color * (1 - Math.abs(player.layer - this.layer) * 0.85),
  );
  this.color = `rgba(${color}, ${color}, ${color}, ${this.alpha})`;
} else {
  this.colorReset();
}

if (
  Math.round(this.x - player.globalOffsetX + this.width) < 0 ||
  Math.round(this.x - player.globalOffsetX) > gameArea.width ||
  Math.round(this.y - player.globalOffsetY + this.height) < 0 ||
  Math.round(this.y - player.globalOffsetY) > gameArea.height ||
  this.alpha <= 0
) {
  this.visible = false;
} else {
  this.visible = true;
}

// Skip rendering if not visible or fully transparent
if (this.visible && this.alpha > 0) {
  if (webGl && !webGl3d) {
    let index =
      Object.keys(enemyTypes).indexOf(this.type) +
      Object.keys(blockTypes).length;
    renderRect(
      Math.round(this.x - player.globalOffsetX),
      Math.round(this.y - player.globalOffsetY),
      this.width,
      this.height,
      index,
      this.alpha,
    );
```

```
      } else if (!webGl && !webGl3d) {
        ctx.fillStyle = this.color;
        ctx.fillRect(
          Math.round(this.x - player.globalOffsetX),
          Math.round(this.y - player.globalOffsetY),
          this.width,
          this.height,
        );
      } else if (webGl3d) {
        if (!this.cube) {
          //this.cube = createColoredCube(this.colorR, this.colorG, this.colorB);
          this.cube = createModel(this.model);
        } else {
          this.cube.visible = true;
        }
        this.cube.position.x = Math.round(this.x - player.globalOffsetX);
        this.cube.position.y = Math.round(this.y - player.globalOffsetY) +
this.height / 2;
        this.cube.position.z = this.layer * 50;
        this.cube.scale.set(this.width / 4, -this.height / 4, this.width / 4);
        setOpacity(this.cube, this.alpha);
      }
    } else if (webGl3d) {
      this.visible = true;
    }
    this.alpha = Math.max(0.1, 1 - Math.abs(player.layer - this.layer) * 0.85);
  }

  update() {
    if (this.movement === "horizontalPatrol") {
      this.horizontalPatrol();
    } else if (this.movement === "verticalPatrol") {
      this.verticalPatrol();
    }
  }

  getSolidBlocksOnLayer() {
    const solids = [];
    const layerData = activeLevelData[currentArea][this.layer];
    // Loop through the rows of the layer
    for (let i = 0; i < layerData.length; i++) {
      // Loop through the columns of the layer
      for (let j = 0; j < layerData[i].length; j++) {
        const object = layerData[i][j];
        if (object.isBlock && object.solid) {
```

```
          solids.push(object);
        }
      }
    }
    return solids;
  }

  // ↓ I'm Probably going to use this as my part of the AP project ↓
  collidesWithSolid(nextX, nextY, solids) {
    for (let i = 0; i < solids.length; i++) {
      //goes through every solid block on the enemy's layer and checks if the
enemy's next position would overlap with any of them. If it does, it returns true,
indicating a collision. If it goes through all solids without finding a collision,
it returns false.
      const block = solids[i];// Loop through the solid blocks and check for
collision
      const overlapX =
        nextX < block.x + block.width && nextX + this.width > block.x; // Check
for overlap in x direction
      const overlapY =
        nextY < block.y + block.height && nextY + this.height > block.y; // Check
for overlap in y directions
      if (overlapX && overlapY) {
        return true;
      } // If there's an overlap in both x and y directions, a collision is
detected
    }
    return false;
  }

  hasGroundAhead(nextX, solids) {
    const lookAheadX = nextX + (this.direction > 0 ? this.width : 0);
    const feetY = this.y + this.height + 2;

    for (let i = 0; i < solids.length; i++) {
      const block = solids[i];
      const overBlockX =
        lookAheadX >= block.x && lookAheadX <= block.x + block.width;
      const nearTopSurface =
        feetY >= block.y && feetY <= block.y + block.height;
      if (overBlockX && nearTopSurface) {
        return true;
      }
    }
```

```
    return false;
  }

  horizontalPatrol() {
    const solids = this.getSolidBlocksOnLayer();
    const nextX = this.x + this.speed * this.direction;

    const willHitWall = this.collidesWithSolid(nextX, this.y, solids);
    const hasFloorAhead = this.hasGroundAhead(nextX, solids);

    if (willHitWall || !hasFloorAhead) {
      this.direction *= -1;
    }

    this.x += this.speed * this.direction;
  }

  verticalPatrol() {
    const solids = this.getSolidBlocksOnLayer();
    const nextY = this.y + this.speed * this.direction;

    const willHitWall = this.collidesWithSolid(this.x, nextY, solids);

    if (willHitWall) {
      this.direction *= -1;
    }

    this.y += this.speed * this.direction;
  }

  remove() {
    if (webGl3d && this.cube) {
      removeObject(this.cube);
      this.cube = null;
    }
  }
}
```

JavaScript

```
//apProjectLevelBuild.js
const levelNumberKey = {
  1: "basic",
  2: "temporary",
```

```
  3: "permanentTemporary",
  4: "ice",
  5: "slow",
  6: "cobweb",
  7: "spike",
}
function convertToObjects(levelData, layer, area, player) {
  let longestHorizontalLength = Math.max.apply(null, levelData.map(row =>
row.length));
  let level = [];
  for (let i = 0; i < levelData.length; i++) {
    let row = [];
    for (let j = 0; j < levelData[i].length; j++) {
      if (levelData[i][j] === "p") {
        player.spawnX = j * 50;
        player.spawnY = i * 50;
        player.spawnLayer = layer;
        player.globalOffsetX = 0;
        player.globalOffsetY = 0;
        player.spawnOffsetX = 0;
        player.spawnOffsetY = 0;
        if (player.spawnX > gameArea.width / 2 && longestHorizontalLength * 50 >
gameArea.width) {
          if (player.spawnX <= longestHorizontalLength * 50 - gameArea.width / 2)
{
            player.spawnX = gameArea.width / 2;
            player.spawnOffsetX = j * 50 - gameArea.width / 2;
          } else {
            let spawnDifference = Math.abs(longestHorizontalLength * 50 -
player.spawnX);
            spawnDifference = Math.floor(spawnDifference / 50) * 50;
            player.spawnX = gameArea.width - spawnDifference;
            player.spawnOffsetX = longestHorizontalLength * 50 - gameArea.width;
          }
        }
        if (player.spawnY > gameArea.height / 2 && levelData.length * 50 >
gameArea.height) {
          if (player.spawnY <= levelData.length * 50 - gameArea.height / 2) {
            player.spawnY = gameArea.height / 2;
            player.spawnOffsetY = i * 50 - gameArea.height / 2;
          } else {
            let spawnDifference = Math.abs(levelData.length * 50 - player.spawnY);
            spawnDifference = Math.floor(spawnDifference / 50) * 50;
            player.spawnY = gameArea.height - spawnDifference;
            player.spawnOffsetY = levelData.length * 50 - gameArea.height;
```

```
          }
        }
        player.spawnValues[area] = [player.spawnX, player.spawnY,
player.spawnLayer, player.spawnOffsetX, player.spawnOffsetY];
      } else if (Object.keys(levelNumberKey).includes(String(levelData[i][j]))) {
        row.push(new Block(j * 50, i * 50, 50, 50, layer,
levelNumberKey[levelData[i][j]]));
      } else if (levelData[i][j][0] === "d") {
        row.push(new Block(j * 50, i * 50, 50, 50, layer, "areaDoorBottom",
levelData[i][j].slice(1)));
      } else if (levelData[i][j][0] === "t") {
        row.push(new Block(j * 50, i * 50, 50, 50, layer, "areaDoorTop",
levelData[i][j].slice(1)));
      } else if (levelData[i][j][0] === "e") {
        row.push(new Enemy(j * 50, i * 50, levelData[i][j].slice(1), layer));
      }
    }
    level.push(row);
  }
  return level;
}

function buildLevel(levelData) {
  for (let i = 0; i < levelData.length; i++) {
    for (let j = 0; j < levelData[i].length; j++) {
      let object = levelData[i][j];
      if (object.isBlock) {
        levelData[i][j].drawBlock();
      } else if (object.isEnemy) {
        levelData[i][j].drawEnemy();
      }
    }
  }
}
```

JavaScript

```
//apProjectLevelData.js
/* Level Number Key:
0 = empty (Any empty lists also work for empty space)
1 = basic block
2 = temporary
3 = permanentTemporary (Doesn't reappear after disappearing)
4 = ice
```

```
5 = slow
6 = cobweb
7 = spike
d = area door bottom (Needs a number after it to determine which area it goes to
(Based off of the index of the area in the level array), ex: d1 would be an area
door that goes to area 1)
t = area door top (Same as area door bottom just a different texture)
e = enemy (Followed by a number for the type of enemy)
*/
const levels = [
  //Level 1
  [
    //Area 1
    [
      //Layer 1
      [
        [],
        [],
        [],
        [1, 1, 2, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1],
      ],
      [
        [],
        [0, 2, 2, 2, 0],
        [0, 0, "p", 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1,
0, 0, 1],
        [0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1],
        [0, 0, 0, 0, 0, 0, 0, 0, 7, 0, 0, 0, "e0", 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
        [1, 1, 1, 1, 1, 2, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1],
        [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, "e1"],
        [0, 0, 1],
        [],
        [0, 1],
        [],
        [0, 0, 1],
        [],
        [0, 1, 0, 0, 0, 0, 6],
        [],
        [0, 0, 1],
        [],
        [0, 1],
        [],
```

```
        [0, 0, 1],
        [],
        [0, 1],
        [],
        [0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, "t1", 0, 0, "t2", 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
        [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, "d1", 0, 0, "d2"],
        [0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1],
        [],
        [0, 0, 1],
        [],
        [0, 1],
        [0, 0, 0],
        [0, 0, 1],
        [],
        [0, 1],
        [],
        [0, 0, 1],
        [0],
        [0, 1],
        [0, 0, 0],
        [0, 0, 1],
        [0, 0],
        [0, 1],
        [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
        [4, 4, 4, 4, 4, 5, 4, 4, 4, 2, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4,
4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4],
      ],
      [
        [],
        [],
        [1, 1, 2, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1],
      ]
    ],
    [
      [
        [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1],
        [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, "d0", 1],
        [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1],
        ["p", 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1],
        [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1],
      ]
    ],
```

```
    [
      [
        [1, 1, 2, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1],
        [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1]
        [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1],
        [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1],
        [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1],
        [0, 0, 0, 0, "d0", 0, 0, 0, 0, 0, 0, 1],
        [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1],
        [1, "p", 1],
        [1, 1, 1]
      ]
    ]
  ],
  [
    [
      [
        [],
        [],
        [],
        [],
        [],
        [],
        [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1],
        [],
        [],
        [],
        [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 1, 1, 1,
1],
        [],
        [],
        ["p"],
        new Array(100).fill(1),
        new Array(100).fill(1),
      ]
    ]
  ],
];
```

JavaScript

```
//apProjectSettings.js
```

```
//PS currently the settings menu has features like framerate which I haven't figured out how to do yet so they just send console.logs when changed, but the keybind and rendering changing works and saves to local storage so you can change your keybinds and rendering method and they will be there when you refresh the page
const panel = document.getElementById("settingsMenu");

// Game Settings Object
const gameSettings = {
	framerate: 60,
	renderMode: "cpu", // "cpu" or "gpu"
  blockImages: true,
	audioVolume: 70,
	audioOutput: "speakers" // "speakers", "headphones", or "mute"
};

// Make gameSettings globally accessible
window.gameSettings = gameSettings;

// Load settings from localStorage
function loadSettings() {
	const saved = localStorage.getItem("apProjectSettings");
	if (saved) {
		const loaded = JSON.parse(saved);
		Object.assign(gameSettings, loaded);
	}
	applySettings();
}

// Save settings to localStorage
function saveSettings() {
	localStorage.setItem("apProjectSettings", JSON.stringify(gameSettings));
}

let webGl = false;
let webGl3d = false;

// Apply settings to the game
function applySettings() {
	// Apply framerate
	const framerateSelect = document.getElementById("framerate");
	if (framerateSelect) {
		framerateSelect.value = gameSettings.framerate;
	}
```

```
        // Apply render mode
        const renderModeSelect = document.getElementById("renderMode");
        if (renderModeSelect) {
                renderModeSelect.value = gameSettings.renderMode;
    if (gameSettings.renderMode === "gpu") {
      canvas.style.display = "none";
      webGlCanvas.style.display = "block";
      webGl3dCanvas.style.display = "none";
      webGl = true;
      webG3d = false;
    } else if (gameSettings.renderMode === "3D") {
      canvas.style.display = "block";
      webGlCanvas.style.display = "none";
      //webGl3dCanvas.style.display = "block";
      webGl3d = true;
    } else {
      canvas.style.display = "block";
      webGlCanvas.style.display = "none";
      webGl3dCanvas.style.display = "none";
      webGl = false;
      webGl3d = false;
    }
        }

  const blockImagesSelect = document.getElementById("blockImages");
  if (blockImagesSelect) {
    blockImagesSelect.value = gameSettings.blockImages ? "on" : "off";
  }

        // Apply audio volume
        const volumeSlider = document.getElementById("audioVolume");
        if (volumeSlider) {
                volumeSlider.value = gameSettings.audioVolume;
                updateVolumeDisplay();
        }

        // Apply audio output
        const audioOutputSelect = document.getElementById("audioOutput");
        if (audioOutputSelect) {
                audioOutputSelect.value = gameSettings.audioOutput;
        }
}

// I still need to fix this so that it actually changes the framerate instead of
just sending console logs
```

```
      const framerateSelect = document.getElementById("framerate");
      if (framerateSelect) {
            framerateSelect.addEventListener("change", (e) => {
                  gameSettings.framerate = parseInt(e.target.value);
                  saveSettings();
                  console.log(`Framerate set to ${gameSettings.framerate} FPS`);
            });
      }

      const renderModeSelect = document.getElementById("renderMode");
      if (renderModeSelect) {
            renderModeSelect.addEventListener("change", (e) => {
                  gameSettings.renderMode = e.target.value;
                  saveSettings();
     if (gameSettings.renderMode === "gpu") {
       canvas.style.display = "none";
       webGlCanvas.style.display = "block";
       webGl3dCanvas.style.display = "none";
       webGl = true;
       webG3d = false;
     } else if (gameSettings.renderMode === "3D") {
       canvas.style.display = "none";
       webGlCanvas.style.display = "none";
       webGl3dCanvas.style.display = "block";
       webGl3d = true;
     } else {
       canvas.style.display = "block";
       webGlCanvas.style.display = "none";
       webGl3dCanvas.style.display = "none";
       webGl = false;
       webGl3d = false;
     }
            });
      }

  const blockImagesSelect = document.getElementById("blockImages");
  if (blockImagesSelect) {
    blockImagesSelect.addEventListener("change", (e) => {
      gameSettings.blockImages = e.target.value === "on";
      saveSettings();
    });
  }


// Setup audio settings listeners
```

```
function setupAudioSettings() {
	const volumeSlider = document.getElementById("audioVolume");
	if (volumeSlider) {
		volumeSlider.addEventListener("input", (e) => {
			gameSettings.audioVolume = parseInt(e.target.value);
			updateVolumeDisplay();
			saveSettings();
		});
	}

	const audioOutputSelect = document.getElementById("audioOutput");
	if (audioOutputSelect) {
		audioOutputSelect.addEventListener("change", (e) => {
			gameSettings.audioOutput = e.target.value;
			saveSettings();
			console.log(`Audio output set to
${gameSettings.audioOutput}`);
		});
	}
}

// Update volume display value
function updateVolumeDisplay() {
	const volumeValue = document.getElementById("volumeValue");
	if (volumeValue) {
		volumeValue.textContent = gameSettings.audioVolume + "%";
	}
}

//Dragging functionality taken from online tutorial for another project then
ported here
function dragElement(elmnt) {
  var pos1 = 0,
    pos2 = 0,
    pos3 = 0,
    pos4 = 0;
  if (document.getElementById(elmnt.id + "header")) {
    // if present, the header is where you move the DIV from:
    document.getElementById(elmnt.id + "header").onmousedown = dragMouseDown;
  } else {
    // otherwise, move the DIV from anywhere inside the DIV:
    elmnt.onmousedown = dragMouseDown;
  }

  function dragMouseDown(e) {
```

```
  e = e || window.event;
  if (e.target.closest(".settings-slider-container") ||
      e.target.closest(".settings-select") ||
      e.target.closest(".keybind") ||
      e.target.closest(".editor-button") ||
      e.target.closest("select") ||
      e.target.closest("button")) {
    return;
  }

  e.preventDefault();
  pos3 = e.clientX;
  pos4 = e.clientY;
  document.onmouseup = closeDragElement;
  document.onmousemove = elementDrag;
}

function elementDrag(e) {
  elmnt.style.position = "absolute";
  e = e || window.event;
  e.preventDefault();

  pos1 = pos3 - e.clientX;
  pos2 = pos4 - e.clientY;
  pos3 = e.clientX;
  pos4 = e.clientY;

  var parentWidth = window.innerWidth;
  var parentHeight = window.innerHeight;
  var newRight = parentWidth - (elmnt.offsetLeft + elmnt.offsetWidth) + pos1;
  var newBottom =
    parentHeight - (elmnt.offsetTop + elmnt.offsetHeight) + pos2;

  // Apply the styles
  elmnt.style.right = newRight + "px";
  elmnt.style.bottom = newBottom + "px";

  // Clear top/left so they don't fight with right/bottom
  elmnt.style.top = "auto";
  elmnt.style.left = "auto";

  elmnt.style.position = "fixed";
}

function closeDragElement() {
```

```
    // stop moving when mouse button is released:
    document.onmouseup = null;
    document.onmousemove = null;
  }
}

dragElement(panel);

const levelEditor = document.getElementById("levelEditorMenu");
dragElement(levelEditor);
const editLevelMenu = document.getElementById("editLevelMenu");
dragElement(editLevelMenu);

function settingsKeybind(buttonId) {
  const button = document.getElementById(buttonId);
  button.textContent = "Press a key...";
  function keyListener(e) {
    if (e.key === "Escape") {
      if (isLetter(controller[buttonId].key[0])) {
        button.textContent = controller[buttonId].key[0].toUpperCase();
      } else if (controller[buttonId].key[0] === " ") {
        button.textContent = "Space";
      } else {
        button.textContent = controller[buttonId].key[0];
      }
      window.removeEventListener("keydown", keyListener);
      return;
    }
    if (isLetter(e.key)) {
      button.textContent = e.key.toUpperCase();
    } else if (e.key === " ") {
      button.textContent = "Space";
    } else {
      button.textContent = e.key;
    }
    updateKeybind(buttonId, e.key);
    window.removeEventListener("keydown", keyListener);
  }
  window.addEventListener("keydown", keyListener);
}

function attachSettingsListeners(buttonId) {
  const button = document.getElementById(buttonId);
  button.textContent = controller[buttonId].key[0];
  button.addEventListener("click", (e) => {
```

```
    e.target.blur();
    settingsKeybind(buttonId);
  });
  window.addEventListener("keydown", (e) => {
    if (e.code === "Space") {
      e.preventDefault();
    }
  });
}

function setupControllerSettings() {
  for (const controllerKey in controller) {
    attachSettingsListeners(controllerKey);
  }
}

// Initialize all settings when the page loads
function initializeSettings() {
       loadSettings();
       setupAudioSettings();
       setupControllerSettings();
}
```

JavaScript

```
// apProjectLevelEditor.js

let levelDataInput = null;

function createNewLevel() {
  const editLevelMenu = document.getElementById("editLevelMenu");
  if (editLevelMenu.classList.contains("hidden")) {
    editLevelMenu.classList.remove("hidden");
    clearEditLevelMenu();
  }
}

function loadLevel() {
  if (levelDataInput) {
    try {
      const jsonData = JSON.parse(levelDataInput);
      if (Array.isArray(jsonData) && jsonData.every((area) =>
Array.isArray(area))) {
        levels.push(jsonData);
```

```
        currentLevel = levels.length - 1;
        updateLevelData();
      } else {
        alert("Invalid level data format. Please provide a 4D array.");
      }
    } catch (error) {
      alert("Error parsing level data. Please provide valid JSON." +
error.message);

    }
  }
}

async function exportLevel() {
  const levelData = JSON.stringify(levels[currentLevel]);
  const clipboardItem = new ClipboardItem({ "text/plain": new Blob([levelData], {
type: "text/plain" }) });
  await navigator.clipboard.write([clipboardItem]);
}

function importLevel() {
  levelDataInput = prompt("Paste the level data (4D array in JSON format):");
}

function closeEditLevelMenu() {
  document.getElementById("editLevelMenu").classList.add("hidden");
  clearEditLevelMenu();
}

function clearEditLevelMenu() {
  document.getElementById("areasContainer").innerHTML = "";
  newArea();
  newLayer(0);
  newRow(0, 0);
  newTile(0, 0, 0);
}

function newArea() {
  const newArea = document.createElement("div");
  const areas = document.querySelectorAll(`#areasContainer > .area`).length;
  newArea.id = `area${areas}`;
  newArea.classList.add("area");
  newArea.innerHTML = `
    <h3 onClick="hide('area${areas}-controls')">Area ${areas + 1}</h3>
    <div id="area${areas}-controls">
```

```
      <label for="newLayerButton${areas}">New Layer:</label>
      <button id="newLayerButton${areas}" class="editor-button"
onclick="newLayer(${areas})">+</button>
    </div>
  `;
  document.getElementById("areasContainer").appendChild(newArea);
  if (areas !== 0) {
    newLayer(areas);
    newRow(areas, 0);
    newTile(areas, 0, 0);
  }
}

function newLayer(areaIndex) {
  const newLayer = document.createElement("div");
  const layerCount = document.querySelectorAll(`#area${areaIndex}-controls >
div`).length;
  newLayer.id = `area${areaIndex}-layer${layerCount}`;
  newLayer.classList.add("layer");
  newLayer.innerHTML = `
    <h4 onClick="hide('area${areaIndex}-layer${layerCount}-controls')">Layer
${areaIndex + 1}.${layerCount + 1}</h4>
    <div id="area${areaIndex}-layer${layerCount}-controls">
      <label for="newRowButton${areaIndex}">New Row:</label>
      <button id="newRowButton${areaIndex}" class="editor-button"
onclick="newRow(${areaIndex}, ${layerCount})">+</button>
    </div>
  `;
  document.getElementById(`area${areaIndex}-controls`).appendChild(newLayer);
  if (layerCount !== 0) {
    newRow(areaIndex, layerCount);
    newTile(areaIndex, layerCount, 0);
  }
}

function newRow(areaIndex, layerIndex) {
  const newRow = document.createElement("div");
  const rowCount =
document.querySelectorAll(`#area${areaIndex}-layer${layerIndex}-controls >
div`).length;
  newRow.id = `area${areaIndex}-layer${layerIndex}-row${rowCount}`;
  newRow.classList.add("row");
  newRow.innerHTML = `
```

```
    <h5
onClick="hide('area${areaIndex}-layer${layerIndex}-row${rowCount}-controls')">Row
${areaIndex + 1}.${layerIndex + 1}.${rowCount + 1}</h5>
    <div id="area${areaIndex}-layer${layerIndex}-row${rowCount}-controls">
      <label for="newTileButton${areaIndex}">New Tile:</label>
      <button id="newTileButton${areaIndex}" class="editor-button"
onclick="newTile(${areaIndex}, ${layerIndex}, ${rowCount})">+</button>
    </div>
  `;

document.getElementById(`area${areaIndex}-layer${layerIndex}-controls`).appendChil
d(newRow);
  if (rowCount !== 0) {
    newTile(areaIndex, layerIndex, rowCount);
  }
}

function newTile(areaIndex, layerIndex, rowIndex) {
  const newTile = document.createElement("div");
  const tileCount =
document.querySelectorAll(`#area${areaIndex}-layer${layerIndex}-row${rowIndex}-con
trols > div`).length;
  newTile.id =
`area${areaIndex}-layer${layerIndex}-row${rowIndex}-tile${tileCount}`;
  newTile.classList.add("tile");
  newTile.innerHTML = `
  <h6
onClick="hide('area${areaIndex}-layer${layerIndex}-row${rowIndex}-tile${tileCount}
-controls')">Tile ${areaIndex + 1}.${layerIndex + 1}.${rowIndex + 1}.${tileCount +
1}</h6>
  <div
id="area${areaIndex}-layer${layerIndex}-row${rowIndex}-tile${tileCount}-controls">
    <label
for="tileTypeInput${areaIndex}-${layerIndex}-${rowIndex}-${tileCount}">Tile:</labe
l>
    <select onclick="tileOptions(this)" class="tileInput"
id="tileTypeInput${areaIndex}-${layerIndex}-${rowIndex}-${tileCount}">
      <option value="0">Empty</option>
    </select>
  </div>
  `;

document.getElementById(`area${areaIndex}-layer${layerIndex}-row${rowIndex}-contro
ls`).appendChild(newTile);
}
```

```
function tileOptions(selectElement) {
  const value = selectElement.value;
  selectElement.innerHTML = `
    <option value="0">Empty</option>
    <option value="1">Basic</option>
    <option value="2">Temporary</option>
    <option value="3">Permanent Temporary</option>
    <option value="4">Ice</option>
    <option value="5">Slow</option>
    <option value="6">Cobweb</option>
    <option value="7">Spike</option>
    <option value="p">Player Spawn</option>`
  ;
  for (let e = 0; e < Object.keys(enemyTypes).length; e++) {
    const enemyOption = document.createElement("option");
    enemyOption.value = `e${e}`;
    enemyOption.textContent = enemyTypes[e].name;
    selectElement.appendChild(enemyOption);
  }
  for (let a = 0; a < document.querySelectorAll(`#areasContainer > .area`).length;
a++) {
    const areaOption = document.createElement("option");
    areaOption.value = `d${a}`;
    areaOption.textContent = `Door to Area ${a + 1}`;
    selectElement.appendChild(areaOption);
  }
  selectElement.value = value;
}

function hide(id) {
  const element = document.getElementById(id);
  if (element.classList.contains("hidden")) {
    element.classList.remove("hidden");
  } else {
    element.classList.add("hidden");
  }
}

function submitLevel() {
  const levelData = [];
  for (let a = 0; a < document.querySelectorAll(`#areasContainer > .area`).length;
a++) {
    const areaData = [];
    const layers = document.querySelectorAll(`#area${a}-controls > .layer`);
```

```
    layers.forEach((layer, l) => {
      const layerData = [];
      const rows = layer.querySelectorAll(`.row`);
      rows.forEach((row, ro) => {
        const rowData = [];
        const tiles = row.querySelectorAll(`.tile`);
        tiles.forEach((tile, t) => {
          const tileType = tile.querySelector("select").value || "0";
          rowData.push(tileType);
        });
        layerData.push(rowData);
      });
      areaData.push(layerData);
    });
    levelData.push(areaData);
  }
  levels.push(levelData);
  currentLevel = levels.length - 1;
  updateLevelData();
  player.spawn();
  closeEditLevelMenu();
}
```

JavaScript

```
// apProject2dWebGl.js
let shadersInitialized = false;

// Shader code adapted from online tutorials
const vertexShaderSourceCode = `#version 300 es
  precision mediump float;

  in vec2 vertexPosition;
  in vec3 vertexColor;

  out vec3 fragmentColor;

  uniform vec2 canvasSize;
  uniform vec2 shapeLocation;
  uniform vec2 shapeSize;

  void main() {
    fragmentColor = vertexColor;
```

```
    vec2 finalVertexPosition = vertexPosition * shapeSize + shapeLocation;
    vec2 clipPosition = (finalVertexPosition / canvasSize) * 2.0 - 1.0;

    gl_Position = vec4(clipPosition * vec2(1.0, -1.0), 0.0, 1.0);
  }
`;

const fragmentShaderSourceCode = `#version 300 es
  precision mediump float;

  in vec3 fragmentColor;
  out vec4 outputColor;
  uniform float uAlpha;

  void main() {
    outputColor = vec4(fragmentColor, uAlpha);
  }
`;

const squareVertices = new Float32Array([
  // Bottom left
  0, 1,
  // Top left
  0, 0,
  // Top right
  1, 0,
  // Bottom left
  0, 1,
  // Top right
  1, 0,
  // Bottom right
  1, 1
]);

const playerVertices = new Float32Array([
  // Bottom left
  0, 1,
  // Top left
  0, 0,
  // Top right
  0.74, 0,
  // Bottom left
  0, 1,
  // Top right
  0.74, 0,
```

```
  // Bottom right
  0.74, 1
]);

function createMonochromeSquareColors(r, g, b) {
  const colors = new Uint8Array([
    r, g, b,
    r, g, b,
    r, g, b,
    r, g, b,
    r, g, b,
    r, g, b
  ]);
  return colors;
}

function createStaticVertexBuffer(gl = WebGL2RenderingContext, data = ArrayBuffer)
{
  const buffer = gl.createBuffer();
  gl.bindBuffer(gl.ARRAY_BUFFER, buffer);
  gl.bufferData(gl.ARRAY_BUFFER, data, gl.STATIC_DRAW);
  gl.bindBuffer(gl.ARRAY_BUFFER, null);
  return buffer;
}

function createTwoBufferVAO(gl = WebGL2RenderingContext, positionBuffer =
WebGLBuffer, colorBuffer = WebGLBuffer, positionAttribLocation = number,
colorAttribLocation = number) {
  const vao = gl.createVertexArray();
  gl.bindVertexArray(vao);
  gl.enableVertexAttribArray(positionAttribLocation);
  gl.enableVertexAttribArray(colorAttribLocation);
  gl.bindBuffer(gl.ARRAY_BUFFER, positionBuffer);
  gl.vertexAttribPointer(positionAttribLocation, 2, gl.FLOAT, false, 0, 0);
  gl.bindBuffer(gl.ARRAY_BUFFER, colorBuffer);
  gl.vertexAttribPointer(colorAttribLocation, 3, gl.UNSIGNED_BYTE, true, 0, 0);
  gl.bindBuffer(gl.ARRAY_BUFFER, null);
  gl.bindVertexArray(null);

  return vao;
}

function webGlBackgroundColor(r, g, b, a) {
  gl.clearColor(r/255, g/255, b/255, a);
  gl.clear(gl.COLOR_BUFFER_BIT | gl.DEPTH_BUFFER_BIT);
```

```
}

function createShapeShaderProgram() {
  const squareGeoBuffer = createStaticVertexBuffer(gl, squareVertices);
  if (!squareGeoBuffer) {
    console.error("Failed to create buffers");
    return null;
  }
  const playerGeoBuffer = createStaticVertexBuffer(gl, playerVertices);
  if (!playerGeoBuffer) {
    console.error("Failed to create buffers");
    return null;
  }
  const squareColorBuffers = [];
  for (let key in blockTypes) {
    let colors = createMonochromeSquareColors(
      blockTypes[key].colorR,
      blockTypes[key].colorG,
      blockTypes[key].colorB
    );
    squareColorBuffers.push(colors);
  }
  for (let key in enemyTypes) {
    let colors = createMonochromeSquareColors(
      enemyTypes[key].colorR,
      enemyTypes[key].colorG,
      enemyTypes[key].colorB
    );
    squareColorBuffers.push(colors);
  }
  squareColorBuffers.push(createMonochromeSquareColors(player.r, player.g,
player.b));

  for (let i = 0; i < squareColorBuffers.length; i++) {
    squareColorBuffers[i] = createStaticVertexBuffer(gl, squareColorBuffers[i]);
    if (!squareColorBuffers[i]) {
      console.error("Failed to create buffers");
      return null;
    }
  }

  const vertexShader = gl.createShader(gl.VERTEX_SHADER);
  gl.shaderSource(vertexShader, vertexShaderSourceCode);
  gl.compileShader(vertexShader);
  if (!gl.getShaderParameter(vertexShader, gl.COMPILE_STATUS)) {
```

```
    console.error("Vertex shader compilation error:",
gl.getShaderInfoLog(vertexShader));
    return null;
  }

  const fragmentShader = gl.createShader(gl.FRAGMENT_SHADER);
  gl.shaderSource(fragmentShader, fragmentShaderSourceCode);
  gl.compileShader(fragmentShader);
  if (!gl.getShaderParameter(fragmentShader, gl.COMPILE_STATUS)) {
    console.error("Fragment shader compilation error:",
gl.getShaderInfoLog(fragmentShader));
    return null;
  }

  const shaderProgram = gl.createProgram();
  gl.attachShader(shaderProgram, vertexShader);
  gl.attachShader(shaderProgram, fragmentShader);
  gl.linkProgram(shaderProgram);
  if (!gl.getProgramParameter(shaderProgram, gl.LINK_STATUS)) {
    console.error("Shader program linking error:",
gl.getProgramInfoLog(shaderProgram));
    return null;
  }

  const positionAttribLocation = gl.getAttribLocation(shaderProgram,
"vertexPosition");
  const colorAttribLocation = gl.getAttribLocation(shaderProgram, "vertexColor");
  if (positionAttribLocation === -1 || colorAttribLocation === -1) {
    console.error("Failed to get attribute locations");
    return null;
  }

  const shapeLocationUniform = gl.getUniformLocation(shaderProgram,
"shapeLocation");
  const shapeSizeUniform = gl.getUniformLocation(shaderProgram, "shapeSize");
  const canvasSizeUniform = gl.getUniformLocation(shaderProgram, "canvasSize");
  const uAlphaUniform = gl.getUniformLocation(shaderProgram, "uAlpha");
  if (!shapeLocationUniform || !shapeSizeUniform || !canvasSizeUniform ||
!uAlphaUniform) {
    console.error("Failed to get uniform locations");
    return null;
  }

  gl.enable(gl.BLEND);
  gl.blendFunc(gl.SRC_ALPHA, gl.ONE_MINUS_SRC_ALPHA);
```

```
  const vaos = [];
  for (let i = 0; i < squareColorBuffers.length; i++) {
    if (i === squareColorBuffers.length - 1) {
      vaos.push(createTwoBufferVAO(gl, playerGeoBuffer, squareColorBuffers[i],
positionAttribLocation, colorAttribLocation));
      break;
    }
    vaos.push(createTwoBufferVAO(gl, squareGeoBuffer, squareColorBuffers[i],
positionAttribLocation, colorAttribLocation));
  }

  return {
    shaderProgram,
    vaos,
    shapeLocationUniform,
    shapeSizeUniform,
    canvasSizeUniform,
    uAlphaUniform
  }
}

const imageVertexShaderSourceCode = `#version 300 es
  precision mediump float;

  in vec2 vertexPosition;
  in vec2 texCoord;

  out vec2 v_texCoord;

  uniform vec2 canvasSize;
  uniform vec2 shapeLocation;
  uniform vec2 shapeSize;

  void main() {
    v_texCoord = texCoord;
    vec2 finalVertexPosition = vertexPosition * shapeSize + shapeLocation;
    vec2 clipPosition = (finalVertexPosition / canvasSize) * 2.0 - 1.0;

    gl_Position = vec4(clipPosition * vec2(1.0, -1.0), 0.0, 1.0);
  }
`;

const imageFragmentShaderSourceCode = `#version 300 es
  precision highp float;
```

```
  uniform sampler2D uTexture;
  uniform float uAlpha;

  in vec2 v_texCoord;

  out vec4 outputColor;

  void main() {
    vec4 color = texture(uTexture, v_texCoord);
    outputColor = vec4(color.rgb, color.a * uAlpha);
  }
`;

const imageTextureCache = new WeakMap();

function createImageShaderProgram() {
  const vertexShader = gl.createShader(gl.VERTEX_SHADER);
  gl.shaderSource(vertexShader, imageVertexShaderSourceCode);
  gl.compileShader(vertexShader);
  if (!gl.getShaderParameter(vertexShader, gl.COMPILE_STATUS)) {
    console.error("Vertex shader compilation error: " +
gl.getShaderInfoLog(vertexShader));
    return;
  }

  const fragmentShader = gl.createShader(gl.FRAGMENT_SHADER);
  gl.shaderSource(fragmentShader, imageFragmentShaderSourceCode);
  gl.compileShader(fragmentShader);
  if (!gl.getShaderParameter(fragmentShader, gl.COMPILE_STATUS)) {
    console.error("Fragment shader compilation error: " +
gl.getShaderInfoLog(fragmentShader));
    return;
  }

  const shaderProgram = gl.createProgram();
  gl.attachShader(shaderProgram, vertexShader);
  gl.attachShader(shaderProgram, fragmentShader);
  gl.linkProgram(shaderProgram);
  if (!gl.getProgramParameter(shaderProgram, gl.LINK_STATUS)) {
    console.error("Shader program linking error: " +
gl.getProgramInfoLog(shaderProgram));
    return;
  }
```

```
  const vertexPositionAttributeLocation = gl.getAttribLocation(shaderProgram,
"vertexPosition");
  const texCoordAttributeLocation = gl.getAttribLocation(shaderProgram,
"texCoord");
  if (vertexPositionAttributeLocation < 0 || texCoordAttributeLocation < 0) {
    console.error("Failed to get the attribute location for vertexPosition or
texCoord");
  }
  const canvasSizeUniform = gl.getUniformLocation(shaderProgram, "canvasSize");
  const shapeLocationUniform = gl.getUniformLocation(shaderProgram,
"shapeLocation");
  const shapeSizeUniform = gl.getUniformLocation(shaderProgram, "shapeSize");
  const uTextureUniform = gl.getUniformLocation(shaderProgram, "uTexture");
  const uAlphaUniform = gl.getUniformLocation(shaderProgram, "uAlpha");
  if (canvasSizeUniform === null || shapeLocationUniform === null ||
shapeSizeUniform === null || uTextureUniform === null || uAlphaUniform === null) {
    console.error("Failed to get uniform locations");
    return;
  }

  const vao = gl.createVertexArray();
  gl.bindVertexArray(vao);

  // Vertex position buffer
  var vertexBuffer = gl.createBuffer();
  if (!vertexBuffer) {
    console.error("Failed to create vertex buffer");
    gl.bindVertexArray(null);
    return;
  }
  gl.bindBuffer(gl.ARRAY_BUFFER, vertexBuffer);
  gl.bufferData(gl.ARRAY_BUFFER, new Float32Array([
     0, 0,
     1, 0,
     0, 1,
     0, 1,
     1, 0,
     1, 1
  ]), gl.STATIC_DRAW);
  gl.enableVertexAttribArray(vertexPositionAttributeLocation);
  gl.vertexAttribPointer(vertexPositionAttributeLocation, 2, gl.FLOAT, false, 0,
0);

  // Texture coordinate buffer
  var texCoordBuffer = gl.createBuffer();
```

```
  if (!texCoordBuffer) {
    console.error("Failed to create texture coordinate buffer");
    gl.bindVertexArray(null);
    return;
  }
  gl.bindBuffer(gl.ARRAY_BUFFER, texCoordBuffer);
  gl.bufferData(gl.ARRAY_BUFFER, new Float32Array([
    0, 0,
    1, 0,
    0, 1,
    0, 1,
    1, 0,
    1, 1
  ]), gl.STATIC_DRAW);
  gl.enableVertexAttribArray(texCoordAttributeLocation);
  gl.vertexAttribPointer(texCoordAttributeLocation, 2, gl.FLOAT, false, 0, 0);

  gl.bindBuffer(gl.ARRAY_BUFFER, null);
  gl.bindVertexArray(null);

  return {
    shaderProgram,
    vao,
    canvasSizeUniform,
    shapeLocationUniform,
    shapeSizeUniform,
    uTextureUniform,
    uAlphaUniform
  };
}

function webGLRender2D() {
  gl.viewport(0, 0, webGlCanvas.width, webGlCanvas.height);
  webGlBackgroundColor(173, 216, 230, 1);
}

shapeShaderProgramInfo = null;
imageShaderProgramInfo = null;

function initializeWebGL() {
  shapeShaderProgramInfo = createShapeShaderProgram();
  imageShaderProgramInfo = createImageShaderProgram();
  if (!shapeShaderProgramInfo || !imageShaderProgramInfo) {
    console.error("Failed to initialize shader program");
    return;
```

```
  } else {shadersInitialized = true;}
}

function renderRect(x, y, width, height, vaoIndex, alpha = 1.0) {
  gl.useProgram(shapeShaderProgramInfo.shaderProgram);
  gl.uniform2f(shapeShaderProgramInfo.canvasSizeUniform, webGlCanvas.width,
webGlCanvas.height);
  gl.uniform2f(shapeShaderProgramInfo.shapeLocationUniform, x, y);
  gl.uniform2f(shapeShaderProgramInfo.shapeSizeUniform, width, height);
  gl.uniform1f(shapeShaderProgramInfo.uAlphaUniform, alpha);

  gl.bindVertexArray(shapeShaderProgramInfo.vaos[vaoIndex]);
  gl.drawArrays(gl.TRIANGLES, 0, 6);
  gl.bindVertexArray(null);
}

function renderImage(image, x, y, width, height, alpha = 1.0) {
  if (!image) return;
  if ("complete" in image && !image.complete) return;

  let texture = imageTextureCache.get(image);
  if (!texture) {
    texture = gl.createTexture();
    if (!texture) return;

    gl.bindTexture(gl.TEXTURE_2D, texture);
    gl.texParameteri(gl.TEXTURE_2D, gl.TEXTURE_WRAP_S, gl.CLAMP_TO_EDGE);
    gl.texParameteri(gl.TEXTURE_2D, gl.TEXTURE_WRAP_T, gl.CLAMP_TO_EDGE);

    gl.pixelStorei(gl.UNPACK_FLIP_Y_WEBGL, false);
    gl.texImage2D(gl.TEXTURE_2D, 0, gl.RGBA, gl.RGBA, gl.UNSIGNED_BYTE, image);

    gl.hint(gl.GENERATE_MIPMAP_HINT, gl.NICEST);
    gl.generateMipmap(gl.TEXTURE_2D);

    gl.texParameteri(gl.TEXTURE_2D, gl.TEXTURE_MIN_FILTER,
gl.NEAREST_MIPMAP_LINEAR);
    gl.texParameteri(gl.TEXTURE_2D, gl.TEXTURE_MAG_FILTER, gl.NEAREST);

    imageTextureCache.set(image, texture);
  }


  gl.viewport(0, 0, webGlCanvas.width, webGlCanvas.height);
```

```
  gl.useProgram(imageShaderProgramInfo.shaderProgram);
  gl.bindVertexArray(imageShaderProgramInfo.vao);

  gl.uniform2f(imageShaderProgramInfo.canvasSizeUniform, webGlCanvas.width,
webGlCanvas.height);
  gl.uniform2f(imageShaderProgramInfo.shapeLocationUniform, x, y);
  gl.uniform2f(imageShaderProgramInfo.shapeSizeUniform, width, height);
  gl.uniform1f(imageShaderProgramInfo.uAlphaUniform, alpha);

  gl.activeTexture(gl.TEXTURE0);
  gl.bindTexture(gl.TEXTURE_2D, texture);

  gl.uniform1i(imageShaderProgramInfo.uTextureUniform, 0);

  gl.drawArrays(gl.TRIANGLES, 0, 6);

  gl.bindVertexArray(null);
}
```

JavaScript

```
// apProjectThree.js
//THREE.JS SETUP
const scene = new THREE.Scene();

const world = new THREE.Group();
world.scale.set(1, -1, 1);
scene.add(world);

// --- GLTF import ---
const gltfLoader = new THREE.GLTFLoader();

// IMPORTANT: set this path to where the file is hosted relative to the HTML page
// Example if your model is at: ./3D-Models/myModel.glb
const toRenderY = (y) => -y;

const camera = new THREE.PerspectiveCamera(75, window.innerWidth /
window.innerHeight, 0.1, 5000);

const renderer = new THREE.WebGLRenderer();
renderer.setSize(window.innerWidth, window.innerHeight);
document.body.appendChild(renderer.domElement);
renderer.domElement.id = "webGl3DCanvas";
```

```
const geometry = new THREE.BoxGeometry(50, 50, 50);

const loader = new THREE.TextureLoader();

const ambientLight = new THREE.AmbientLight(0x404040, 3);
scene.add(ambientLight);

function createColoredCube(r, g, b) {
  const material = new THREE.MeshPhongMaterial({color: new THREE.Color(r/255,
g/255, b/255)});
  const cube = new THREE.Mesh(geometry, material);
  world.add(cube);
  return cube;
}

function createTexturedCube(image) {
  const texture = new THREE.Texture(image);
  texture.needsUpdate = true;
  texture.colorSpace = THREE.SRGBColorSpace;
  const material = new THREE.MeshPhongMaterial({map: texture});
  const cube = new THREE.Mesh(geometry, material);
  world.add(cube);
  return cube;
}

function createModel(modelPath) {
  const modelObject = new THREE.Group();
  world.add(modelObject);

  gltfLoader.load(
    modelPath,
    (gltf) => {
      const model = gltf.scene;
      modelObject.add(model);

      model.traverse((obj) => {
        if (obj.isMesh) {
          obj.castShadow = true;
          obj.receiveShadow = true;
        }
      });
    },
    undefined,
    (err) => {
      console.error('GLTF failed to load:', err);
```

```
    }
  );
  return modelObject;
}

function setColor(object, r, g, b) {
  let objectColor = new THREE.Color(r/255, g/255, b/255);
  object.material.dispose();
  if (object.material) {
    object.material.color = objectColor;
    object.material.map = null;
  }
}

function setTexture(object, image) {
  const texture = new THREE.Texture(image);
  texture.needsUpdate = true;
  texture.colorSpace = THREE.SRGBColorSpace;
  texture.minFilter = THREE.NearestMipmapLinearFilter;
  texture.magFilter = THREE.NearestFilter;
  object.material.dispose();
  if (object.material) {
    object.material.map = texture;
    object.material.color = new THREE.Color(1, 1, 1);
  }
}

function setOpacity(object, opacity) {
  if (object.material) {
    object.material.transparent = opacity < 1;
    object.material.opacity = opacity;
  }
}

function sceneBackgroundColor(r, g, b) {
  scene.background = new THREE.Color(r/255, g/255, b/255);
}

function sceneBackgroundTexture(texture) {
  scene.background = texture;
}

function removeObject(object) {
  if (object) {
    world.remove(object);
```

```
    if (object.geometry) object.geometry.dispose();
    if (object.material) {
      if (object.material.map) object.material.map.dispose();
      object.material.dispose();
    }
  }
}

webGl3DCanvas.addEventListener("click", () => {
  if (gameRunning) {
    webGl3DCanvas.requestPointerLock();
  }
});

let cameraYaw = 0;
let cameraPitch = 0;

const mouseSensitivity = 0.002;

window.document.addEventListener("mousemove", (e) => {
  if (document.pointerLockElement !== renderer.domElement) return;

  cameraYaw -= e.movementX * mouseSensitivity;
  cameraPitch -= e.movementY * mouseSensitivity;

  const maxPitch = Math.PI / 2 - 0.01;
  if (cameraPitch > maxPitch) cameraPitch = maxPitch;
  if (cameraPitch < -maxPitch) cameraPitch = -maxPitch;
});

function renderScene() {
  renderer.setSize(window.innerWidth, window.innerHeight);

  if (!player.dead) {
    sceneBackgroundColor(173, 216, 230);
  }

  camera.aspect = window.innerWidth / window.innerHeight;
  camera.updateProjectionMatrix();

  camera.position.set(player.x, toRenderY(player.y), player.z);

  camera.rotation.order = "YXZ";
  camera.rotation.y = cameraYaw;
  camera.rotation.x = cameraPitch;
```

```
  renderer.render(scene, camera);
}
```